Machine Learning 2024-1 Proyecto 4

Luis Enrique Cortijo Gonzales 100 %,
Davi Magalhaes Eler 100 %,
Anderson David Cárcamo Vargas 100 %,
Edgar Chambilla 100 %
Universidad de Ingeniería y Tecnología UTEC
Repositorio del proyecto

***Resumen*—En el presente informe se detalla el proceso, la experimentación y los resultados de la implementación de dos modelos de machine learning para el proyecto final del curso. Estos modelos fueron creados para el caso de *Sentiment Classification in Movie Reviews* utilizando el correspondiente conjunto de datos. Este caso consiste en un dataset con 50,000 reseñas de películas divididas equitativamente en 25,000 reseñas para entrenamiento y 25,000 para prueba, con una distribución equilibrada de etiquetas (25,000 positivas y 25,000 negativas).**

En toda la colección, no se permiten más de 30 reseñas para una misma película porque las reseñas de la misma película tienden a tener calificaciones correlacionadas. Además, los conjuntos de entrenamiento y prueba contienen un conjunto disjunto de películas, por lo que no se obtiene un rendimiento significativo al memorizar términos únicos de películas y sus etiquetas observadas. En los conjuntos de entrenamiento/prueba etiquetados, una reseña negativa tiene una puntuación 4 de 10 o menor, y una reseña positiva tiene una puntuación 7 de 10 o mayor. Por lo tanto, las reseñas con calificaciones más neutras no se incluyen en los conjuntos de entrenamiento/prueba.

Este dataset se empleará con el objetivo de clasificar el sentimiento de las reseñas de películas, proporcionando un entorno desafiante debido a la variabilidad en el lenguaje utilizado por diferentes críticos. El objetivo del proyecto es desarrollar los modelos adecuados en base al conjunto de datos proporcionado, lograr el correcto preprocesamiento de los textos, realizar la reducción de la dimensionalidad correspondiente, entrenar los modelos junto con sus predicciones y, finalmente, mostrar los resultados aplicando métricas y gráficos.

Palabras clave: Sentiment Classification, movie reviews, dataset, machine learning, modelos de machine learning, preprocesamiento, reducción de dimensionalidad, entrenamiento de modelos, predicciones, métricas, experimentación, características del dataset, análisis de resultados, reseñas de películas, clasificación de sentimiento, equilibrio de etiquetas, conjuntos de entrenamiento y prueba.

I. Introducción

En este proyecto se sintetizan todos los conceptos aprendidos durante el semestre en el curso de machine learning y sus aplicaciones para la solución de problemas que requieran procesar una amplia cantidad de datos, impidiendo ser realizadas por el ser humano de manera óptima.

A diferencia de proyectos anteriores, en este proyecto no se parte de una serie de algoritmos específicos; por el contrario, en base al problema, nosotros tenemos que realizar el análisis pertinente y proponer el algoritmo correcto para solucionar el problema.

Luego de realizar el análisis, identificamos que este desafío corresponde a la identificación de acciones humanas mediante las escenas, por lo que entraría dentro de una problemática de clasificación. Por lo tanto, las propuestas de algoritmos que emplearemos serán de clasificación; dentro de estos están los supervisados y los no supervisados.

Definiendo la acción de clasificar en el contexto computacional, previamente es obligatorio llevar los datos al plano matemático para que estos sean capaces de ser procesados por el computador. Ese paso previo se llama preprocesamiento y, en el contexto del proyecto, consistirá en sacar los vectores característicos del dataset. Clasificar consta de agrupar los datos en conjuntos que compartan características distinguibles con los demás. La acción de clasificar es natural en el ser humano y fácilmente comprendible, pero cuando se trabaja con grandes volúmenes de datos o se requieren soluciones en las cuales sea complicado tener recursos humanos, surgen como soluciones los algoritmos de clasificación.

Cómo clasifica cada algoritmo es propio de cada uno, pero todos emplean técnicas matemáticas para distinguir los datos y agruparlos. A este proceso se le llamará entrenamiento. Posteriormente, cuando se reciban nuevos volúmenes de datos, se le llamará predicción.

En síntesis, realizar un modelo de machine learning de clasificación consta de 3 importantes hitos a lograr correctamente: el preprocesamiento, el entrenamiento y la predicción.

I-A. Objetivos

Los objetivos del proyecto son los siguientes:

- Comprender y analizar los modelos para su correcta elección y desarrollo.
- Analizar los datos y decidir qué herramientas y técnicas usar para su reducción de dimensionalidad.
- Desarrollar 2 modelos de machine learning de clasificación, que son los más adecuados para el problema dado.
- Lograr que el proceso de preprocesamiento y entrenamiento sea eficiente computacionalmente, garantizando la escalabilidad del modelo a mayores cargas de datos.
- Obtener una predicción adecuada a partir de nuestros modelos.

I-B. Importancia

Este proyecto cumple un rol destacado sobre los otros proyectos realizados en este semestre por la complejidad su-

perior respecto a los proyectos anteriores, sumado a la libertad para elegir qué modelos realizar. Adicionalmente, no está de más recordar la importancia y posibilidades que brindan los modelos de clasificación a la hora de solucionar problemas, gracias a su procesamiento de grandes volúmenes de datos, ya sea con la identificación de personas u objetos aplicada en vigilancia, seguridad, detección de anomalías, accidentes de tránsito, provisión de respuestas asistidas con inteligencia artificial, entre otras posibilidades que surgen gracias a estos modelos. Por lo que desarrollarlos y comprenderlos abre las posibilidades a la solución de problemas modernos.

II. Dataset

El dataset contiene 50,000 reseñas de películas divididas equitativamente en 25,000 para el conjunto de entrenamiento y 25,000 para el conjunto de prueba. La distribución general de las etiquetas está equilibrada (25,000 positivas y 25,000 negativas). No se permiten más de 30 reseñas para una misma película porque las reseñas de la misma película tienden a tener calificaciones correlacionadas. Además, los conjuntos de entrenamiento y prueba contienen un conjunto disjunto de películas, por lo que no se obtiene un rendimiento significativo al memorizar términos únicos de películas y sus etiquetas observadas. En los conjuntos de entrenamiento/prueba etiquetados, una reseña negativa tiene una puntuación no mayor a 4 de 10, y una reseña positiva tiene una puntuación no menor a 7 de 10. Por lo tanto, las reseñas con calificaciones más neutras no se incluyen en los conjuntos de entrenamiento/prueba.

III. Metodología

III-A. Preprocesamiento

III-A1. nltk: Se empleó NLTK para la tarea de preprocesamiento. Esta librería brinda herramientas de procesamiento de lenguaje natural. En este proyecto, utilizamos la técnica de TF-IDF para preprocesar el texto.

TF-IDF (Term Frequency-Inverse Document Frequency) se compone de los siguientes elementos:

- TF (Frecuencia de Término): Mide la frecuencia de un término en un documento específico. Se calcula como el número de veces que una palabra aparece en un documento dividido por el número total de palabras en ese documento. La fórmula es:

$$\text{TF}(t, d) = \frac{f_{t,d}}{N_d}$$

donde $f_{t,d}$ es la frecuencia del término t en el documento d y N_d es el número total de términos en d.

- IDF (Frecuencia Inversa de Documentos): Mide la importancia de un término en el corpus completo. Se calcula como el logaritmo del número total de documentos en el corpus dividido por el número de documentos que contienen el término. La fórmula es:

$$\text{IDF}(t, D) = \log\left(\frac{N}{d_t}\right)$$

donde N es el número total de documentos y d_t es el número de documentos que contienen el término t.

El valor TF-IDF para un término en un documento se obtiene multiplicando el valor de TF por el valor de IDF:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

Para implementar esta técnica, primero tokenizamos y preprocesamos el texto usando NLTK. La tokenización divide el texto en oraciones y palabras, se eliminan las palabras vacías (stopwords) y se normalizan las palabras, pasando a minúsculas y eliminando caracteres no alfanuméricos.

III-A2. TfidfVectorizer: Esta librería toma el resultado del preprocesamiento anterior y lo convierte en vectores característicos.

El proceso de vectorización mediante TF-IDF (Term Frequency-Inverse Document Frequency) permite transformar el texto en una representación numérica que refleja tanto la importancia de las palabras en un documento específico (TF) como la relevancia de esas palabras en el conjunto completo de documentos (IDF). De este modo, se pueden identificar las palabras que son más significativas para cada documento, lo que facilita el análisis y la clasificación de texto.

Una vez que los textos han sido vectorizados utilizando TfidfVectorizer, los datos se encuentran en un formato estructurado y cuantificable, ideal para ser utilizados en algoritmos de aprendizaje automático.

III-A3. Escalamiento: Para asegurar que todas las características tengan la misma escala y mejorar el rendimiento de los algoritmos de aprendizaje automático, se aplicó el escalamiento de características. Utilizamos el *StandardScaler* de la librería *scikit-learn*, que estandariza las características eliminando la media y escalando a la varianza unitaria. La fórmula es:

$$z = \frac{x - \mu}{\sigma}$$

donde x es el valor original de la característica, μ es la media de la característica y σ es la desviación estándar de la característica.

III-A4. Análisis de Componentes Principales (PCA): Para reducir la dimensionalidad de los datos y eliminar redundancias, se aplicó el Análisis de Componentes Principales (PCA). PCA es una técnica de reducción de dimensionalidad que transforma las características originales en un nuevo conjunto de características no correlacionadas llamadas componentes principales, ordenadas por la cantidad de varianza que explican. Los pasos son:

1. Estandarización de los datos. 2. Cálculo de la matriz de covarianza. 3. Cálculo de los vectores propios y valores propios de la matriz de covarianza. 4. Selección de los principales componentes. 5. Transformación de los datos originales a los nuevos componentes principales.

La fórmula para transformar los datos es:

$$Z = XW$$

donde X es la matriz de datos estandarizados, W es la matriz de vectores propios y Z es la matriz transformada de componentes principales.

El uso de PCA ayuda a reducir el ruido y a mejorar la eficiencia computacional de los modelos de aprendizaje automático.

III-B. Modelos escogidos

III-B1. SVM: SVM (Support Vector Machine por sus siglas en inglés) es un algoritmo de aprendizaje supervisado comúnmente empleado en tareas de clasificación. Las SVM funcionan encontrando el hiperplano óptimo que separa las diferentes clases en el espacio de características. Este hiperplano es elegido de manera que maximice el margen, es decir, la distancia entre el hiperplano y los puntos de datos más cercanos de cada clase, conocidos como vectores de soporte.

Hiperplano El hiperplano en un espacio n-dimensional se puede definir como:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

donde: - $\mathbf{w}$ es un vector de pesos. - $\mathbf{x}$ es el vector de características. - b es un sesgo (bias).

Margen El margen (M) se define como la distancia entre los vectores de soporte y el hiperplano y se calcula como:

$$M = \frac{2}{\|\mathbf{w}\|}$$

Función de Costo La SVM busca minimizar la siguiente función de costo, que incluye un término de regularización y una suma de términos de pérdida de margen:

$$\min_{\mathbf{w},b} \left(\frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{n} \text{máx}(0, 1 - y_i(\mathbf{w} \cdot \mathbf{x}_i + b)) \right)$$

donde: - C es un parámetro de regularización que controla el equilibrio entre maximizar el margen y minimizar el error de clasificación. - y_i son las etiquetas de clase para los ejemplos de entrenamiento. - $\mathbf{x}_i$ son los vectores de características de los ejemplos de entrenamiento.

III-B2. Regresión logística: La Regresión Logística es un modelo de aprendizaje supervisado utilizado principalmente para problemas de clasificación binaria, aunque puede extenderse a clasificaciones multiclase como lo hacemos en este proyecto. A pesar de su nombre, la Regresión Logística se emplea para predecir la probabilidad de que un ejemplo pertenezca a una clase en particular. Los principales componentes para implementar una regresión logistica multiclase son los siguientes: Función Logística

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

donde $z = \mathbf{w}^T\mathbf{x} + b$ es la combinación lineal de las características ($\mathbf{x}$) con los pesos ($\mathbf{w}$) y el sesgo (b) del modelo.

Función de Costo La Regresión Logística utiliza la función de costo conocida como entropía cruzada (log-loss) para penalizar las predicciones incorrectas:

$$J(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

donde: - m es el número de ejemplos de entrenamiento. - y_i son las etiquetas reales de los ejemplos. - $\hat{y}_i = P(y = 1|\mathbf{x}_i)$ son las predicciones del modelo para los ejemplos.

El objetivo es minimizar esta función de costo ajustando los pesos $\mathbf{w}$ y el sesgo b.

III-B3. Red Neuronal Multicapa: Este modelo de red neuronal multicapa está diseñado utilizando la biblioteca PyTorch y se compone de varias capas densamente conectadas (fully connected layers), acompañadas de técnicas adicionales como la normalización por lotes, funciones de activación, y regularización mediante dropout. Sus principales componentes son los siguientes:

Operación Lineal: Cada capa oculta realiza una operación lineal:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

donde $\mathbf{w}$ son los pesos, $\mathbf{x}$ es la entrada, y b es el sesgo.

Normalización por Lotes: La normalización por lotes estandariza las activaciones de una capa:

$$\text{BN}(z) = \gamma \left(\frac{z - \mu}{\sigma} \right) + \beta$$

donde μ y σ son la media y desviación estándar del lote, y γ y β son parámetros aprendibles.

Función de Activación ReLU: La función de activación ReLU se define como:

$$\text{ReLU}(z) = \text{máx}(0, z)$$

Dropout: Dropout desactiva aleatoriamente un porcentaje de neuronas durante el entrenamiento para prevenir el sobreajuste.

Función de Activación Sigmoide: La función sigmoide se utiliza en la capa de salida:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Propagación hacia Adelante

Durante la propagación hacia adelante, las entradas pasan a través de las capas ocultas, donde se aplican las operaciones lineales, normalización por lotes, funciones de activación y dropout. Finalmente, la capa de salida aplica la función sigmoide para producir la probabilidad de clasificación.

IV. Reducción de la Dimensionalidad

Dentro del preprocesamiento de datos, la reducción de la dimensionalidad desempeña un papel fundamental en la mejora del proceso de entrenamiento del modelo. Esto se debe a que surgen tres problemas al utilizar todas las dimensiones de los vectores característicos:

- La inclusión de características irrelevantes que pueden interferir en la clasificación. - La complejidad computacional asociada con el procesamiento de datos de alta dimensionalidad. - El riesgo de sobreajuste (overfitting) en nuestros modelos, lo cual puede resultar en un rendimiento inferior.

Por todas estas razones, se toma la decisión de reducir la dimensionalidad.

IV-A. PCA

Principal Component Analysis (PCA) es la técnica que elegimos para el preprocesamiento de los datos en este proyecto, tambien fue empleado anteriormente en otros proyectos debido a las ventajas que tiene emplearlo. Esta función recibe como parámetros el dataset en vectores, ya sean estandarizados previamente o no, y un número entero k.

Tomando la descripción empleada en un proyecto anterior:

PCA se basa en la construcción de una matriz de covarianza para identificar la relación entre las características. La fórmula para calcular la matriz de covarianza es:

$$\mathbf{C} = \frac{1}{n-1} \sum_{i=1}^{n} (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$$

donde $\mathbf{x}_i$ representa los datos, $\bar{\mathbf{x}}$ es el vector de medias de los datos, y n es el número de muestras. Posteriormente, se procede al cálculo de los valores propios (eigenvalues) y los vectores propios (eigenvectors) a partir de la matriz de covarianza $\mathbf{C}$. La fórmula para este cálculo es:

$$\mathbf{C}\mathbf{v} = \lambda \mathbf{v}$$

donde λ representa los eigenvalues y $\mathbf{v}$ los eigenvectors. Los eigenvalues y eigenvectors se ordenan de mayor a menor, y se seleccionan los primeros k eigenvalues y sus correspondientes eigenvectors. Los datos originales se proyectan sobre estos k vectores propios para obtener los nuevos vectores característicos. La proyección se realiza mediante la siguiente fórmula:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}$$

donde $\mathbf{Z}$ son los datos transformados, $\mathbf{X}$ es la matriz de datos original y $\mathbf{W}$ es la matriz de los primeros k eigenvectors. Esta transformación permite reducir la dimensionalidad de los datos, facilitando su procesamiento y mejorando la capacidad del modelo para generalizar sobre nuevos datos.

IV-B. Justificación

En el proyecto se empleó PCA sobre otras técnicas para la reducción de la dimensionalidad por los siguientes criterios:

- **Eficiencia computacional**: PCA es una técnica lineal en su ejecución, lo que la hace más eficiente que otras técnicas no lineales cuando se utiliza con grandes cantidades de datos.
- **Mejores resultados con los modelos**: PCA proporciona vectores que, según la experimentación realizada, resultaron ser más rápidos en términos computacionales durante el entrenamiento.
- **Mejores predicciones**: Durante la experimentación, PCA arrojó mejores resultados en las predicciones. Esto se debe a la regularización implícita que realiza PCA al devolver los primeros k vectores, lo que evita el sobreajuste (overfitting).

V. Implementación

En primer lugar, se desarrolló el procesamiento de los textos, para lo cual se implementaron un tokenizador, un stemmer y se utilizó un diccionario en inglés. Una vez preprocesados los textos, se procedió a buscar un índice que permitiera caracterizar y asignar un valor a cada uno de ellos. Por esta razón, se optó por utilizar el TF-IDF, el cual permite valorar las palabras características de cada texto.

Como resultado de este proceso, se obtuvo un número adecuado de índices, dado que nuestra matriz de vectores característicos tenía un tamaño de 19,212. Posteriormente, se realizó el escalamiento de los datos utilizando técnicas como el Análisis de Componentes Principales (PCA), con el fin de evaluar la cantidad de vectores que mejor representaban nuestra data. Como se muestra en la imagen, con un número de 11,923 componentes se obtenía una representación del 95 %.

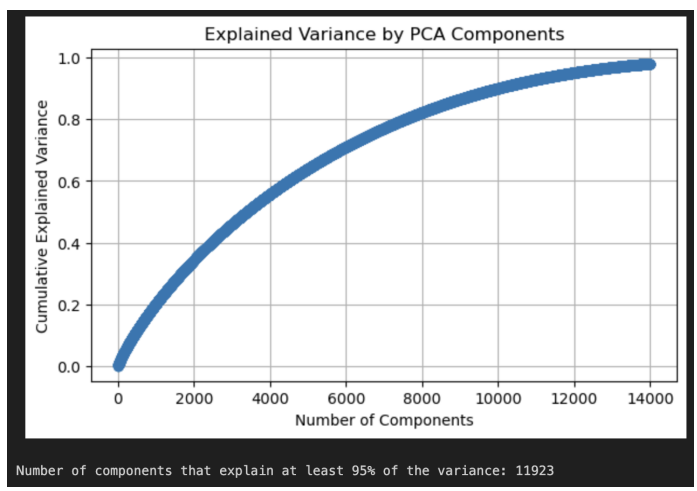

Figura 1. Varianza del PCA.

Finalmente, se evaluó esta matriz característica utilizando los modelos previamente mencionados, como la Máquina de Vectores de Soporte (SVM), regresión logística y redes neuronales.

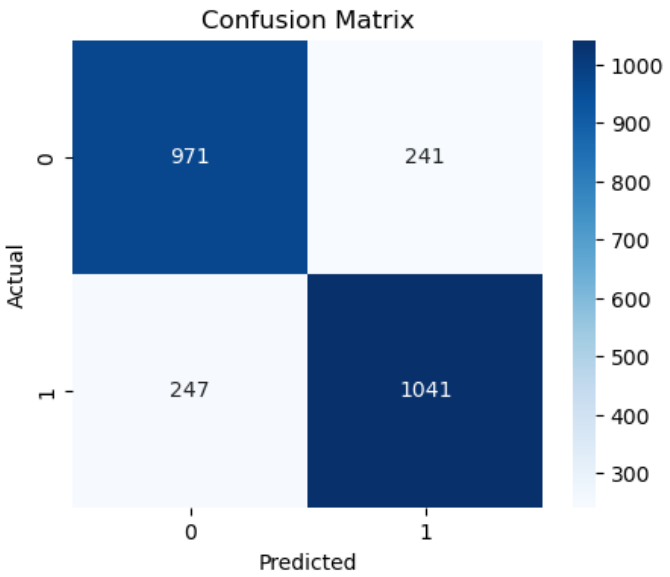

Figura 2. F1 score de regresión logistica.

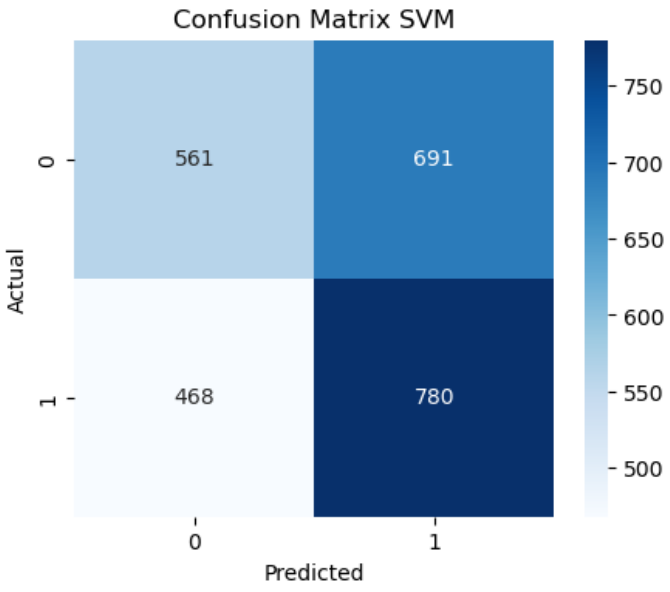

Figura 3. F1 score de SVM.

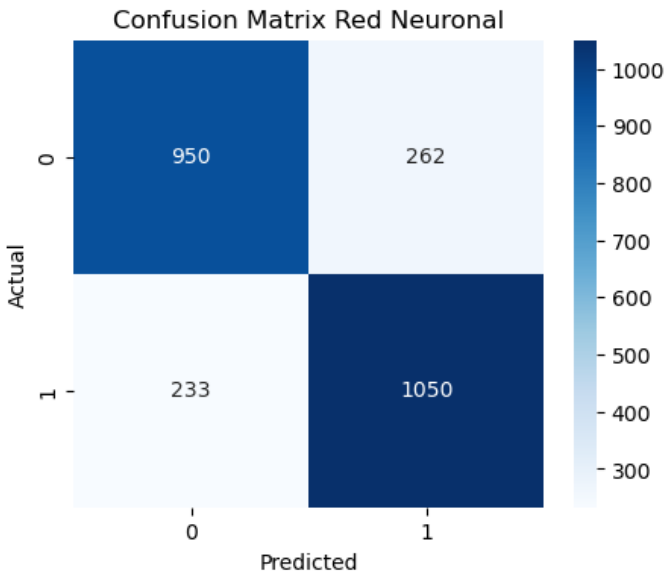

Figura 4. F1 score de Red Neuronal.

VI. Experimentación

En la fase de experimentación, se utilizaron también otros tipos de procesamiento de textos, tales como el tokenizador y el análisis sintáctico (parsing), con el fin de estandarizar el tamaño de los textos. No obstante, aunque este tipo de procesamiento resultó en una matriz característica de menor tamaño, no generó mejores resultados en nuestras pruebas utilizando los mismos modelos de clasificación.

VII. Discusión

VIII. Discusión

Durante el desarrollo del proyecto, se utilizaron diversos modelos de clustering para analizar los datos. Sin embargo, observamos que estos modelos no funcionaron de manera óptima. A pesar de implementar distintos métodos de preprocesamiento de textos, los resultados no mejoraron significativamente.

Es importante destacar que los modelos de clustering pueden requerir ajustes más precisos y datos adicionales para alcanzar un rendimiento óptimo. En futuras investigaciones, sería beneficioso evaluar estos modelos con mayor detalle y considerar técnicas de preprocesamiento avanzadas.

Asimismo, exploramos el uso de redes neuronales para el entrenamiento de nuestros modelos. Los resultados iniciales indican que se necesita un análisis más profundo y posiblemente una arquitectura de red más compleja para obtener mejoras sustanciales en el rendimiento.

En conclusión, aunque los enfoques empleados en este proyecto no lograron los resultados esperados, las lecciones aprendidas proporcionan una base sólida para futuras investigaciones. Se recomienda continuar explorando diferentes técnicas y modelos, así como realizar una evaluación exhaustiva de los parámetros y métodos de preprocesamiento utilizados. Con estos ajustes, es probable que se puedan obtener resultados más prometedores en investigaciones posteriores.

IX. Conclusión

NLTK y TF-IDF: Utilizamos estas técnicas de preprocesamiento para convertir los comentarios de nuestro dataset a vectores característicos numéricos normalizados y listos para ser usados en nuestros modelos. Si bien TF-IDF resultó ser una técnica bastante eficaz para generar vectores característicos, podríamos haber experimentado con otras técnicas.

Modelos utilizados: Fue ventajoso utilizar modelos que trabajan de formas bien distintas para atacar el mismo problema, aunque los resultados obtenidos con cada modelo fueron similares. Además, pudimos poner en práctica nuevamente modelos vistos en distintas semanas del curso.

Reducción de dimensionalidad: El proyecto destacó la importancia de realizar una adecuada reducción de dimensionalidad para poder entrenar nuestros modelos de forma más rápida. PCA demostró buenos resultados.

En resumen, el proyecto demostró que un enfoque integral que combine un buen preprocesamiento, una reducción adecuada de dimensionalidad y la elección adecuada del modelo para lograr una clasificación precisa y eficiente es fundamental. Los hallazgos destacan la capacidad de los modelos de aprendizaje automático para resolver problemas complejos de manera efectiva.